A COLLEGE DOMAIN SPECIFIC MARKETPLACE FOR ACADEMIC RESOURCE REUSE ON CLOUD-NATIVE MICROSERVICE ARCHITECTURE

CLOUD COMPUTING

Submitted by

KAVEESHAA S (230701146)
HARINI P (230701102)
KRITHIKA PV(230701157)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE

NOVEMBER 2025

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled "CLOUD BASED FILE UPLOAD AND SHARING APP " is the bonafide work **KAVEESHAA S (230701146), HARINI P (230701102), KRITHIKA PV(230701157)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. E. M Malathy
Professor &
Head of The Department
Department of Computer Science and Engineering
Rajalakshmi Engineering College

SIGNATURE

Dr.PONMANI
Supervisor
Associate Professor
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on_____

Internal Examiner **External Examiner**

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr.MALATHY E.M, Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **DR. PONMANI** Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

HARINI P
KAVEESHAA S
KRITHIKA PV

ABSTRACT

Cloud computing and DevOps practices have become essential for developing scalable, secure, and reliable software applications. This project focuses on designing and deploying a cloud-based application using Microsoft Azure services integrated with DevOps methodologies. The main objective of the project is to automate application deployment, improve operational efficiency, ensure system security, and provide continuous monitoring through modern cloud technologies.The project implements Infrastructure-as-Code (IaC) using Terraform for automated resource provisioning, Docker for containerization, and Azure Kubernetes Service (AKS) for container orchestration and scaling. Continuous Integration and Continuous Deployment (CI/CD) pipelines are configured to automate code building, testing, and deployment processes. Security practices such as DevSecOps integration, Role-Based Access Control (RBAC), vulnerability scanning, and disaster recovery planning are also included to enhance system reliability and protection.In addition, the project integrates Generative AI services through Azure AI tools to improve intelligent processing and automation capabilities. Monitoring and observability are achieved using Azure Monitor and Application Insights to track system performance, detect issues, and maintain application availability . The proposed solution provides a scalable, cost-effective, and efficient cloud deployment architecture suitable for modern enterprise applications. The project demonstrates practical implementation of cloud operations, DevOps automation, container orchestration, security management, and AI integration in a real-world environment. The final outcome highlights improved deployment speed, better resource management, enhanced security, and simplified maintenance processes. This project also emphasizes the importance of collaboration, automation, and continuous improvement in modern software development environments. By combining cloud computing, DevOps, containerization, Kubernetes orchestration, and AI-based services, the system achieves higher scalability, flexibility, and operational stability. The implementation reduces manual intervention, minimizes deployment errors, and enables faster delivery of application updates. Overall, the project serves as a comprehensive model for building secure, automated, and intelligent cloud-native applications capable of meeting current industry requirements and future technological advancements.

TABLE OF CONTENTS

CHAPTER 1 - INTRODUCTION

1.1 PROBLEM STATEMENT

Organizations today face several challenges in managing modern cloud applications using traditional deployment methods. Existing systems often lack automation, scalability, security, and efficient monitoring capabilities. The major problems identified in the current environment are as follows:

• **Manual Deployment and Configuration Errors:** Traditional deployment processes require manual configuration and infrastructure setup, which increases human errors, application downtime, and maintenance complexity. Repeated manual tasks also reduce operational efficiency and delay software delivery.

• **Lack of Scalability and Resource Management:** Many existing systems are unable to dynamically scale resources based on user demand. This leads to poor performance during high traffic and unnecessary resource consumption during low usage periods.

• **Complex Container and Infrastructure Management:** Managing containers, virtual machines, and cloud resources without orchestration tools becomes difficult as application complexity increases. Infrastructure management without automation also creates inconsistencies across environments.

• **Security and Access Control Challenges:** Ensuring secure access, vulnerability management, and proper user permissions in cloud environments is challenging without integrated DevSecOps practices and Role-Based Access Control (RBAC) mechanisms.

• **Insufficient Monitoring and Failure Detection:** Traditional systems lack real-time monitoring and observability features, making it difficult to identify application failures, performance bottlenecks, and security threats quickly.

• **Need for Faster Software Delivery:** Modern organizations require continuous integration and continuous deployment (CI/CD) pipelines to release updates rapidly and reliably. Existing approaches often slow down development and deployment cycles.

• **Limited Intelligent Automation:** Many systems do not utilize AI-based services for intelligent processing, automation, and analytics, reducing overall operational efficiency and innovation capabilities.

Therefore, there is a strong need for a cloud-native, automated, secure, and scalable solution that integrates DevOps practices, containerization, Kubernetes orchestration, monitoring tools, and Azure AI services to improve overall system performance and reliability.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of this project is to design and implement a secure, scalable, and automated cloud-native application using Microsoft Azure and DevOps technologies. The project aims to improve deployment efficiency, infrastructure management, monitoring, and security through modern cloud solutions. The major objectives of the project are as follows:

• **Automate Infrastructure Deployment:** To implement Infrastructure-as-Code (IaC) using Terraform for automatic provisioning and management of cloud resources, reducing manual effort and configuration errors.

• **Implement Continuous Integration and Continuous Deployment (CI/CD):** To establish automated build, testing, and deployment pipelines that enable faster and reliable software delivery.

• **Enable Containerization:** To use Docker for packaging applications into containers, ensuring portability, consistency, and simplified application deployment across environments.

• **Deploy and Manage Kubernetes Clusters:** To utilize Azure Kubernetes Service (AKS) for orchestrating, scaling, and managing containerized applications efficiently.

• **Improve Security and Access Control:** To integrate DevSecOps practices, vulnerability scanning, and Role-Based Access Control (RBAC) mechanisms for secure cloud operations.

• **Enhance Monitoring and Observability:** To implement Azure Monitor and Application Insights for real-time monitoring, logging, performance tracking, and issue detection.

• **Integrate AI-Based Cloud Services:** To use Azure AI services for intelligent automation and advanced processing capabilities within the application environment.

• **Ensure High Availability and Disaster Recovery:** To implement backup strategies, blue–green deployment methods, and disaster recovery planning for maintaining system reliability and business continuity.

1.3 SCOPE AND BOUNDARIES

The scope of this project includes the design, development, and deployment of a cloud-native application using Microsoft Azure cloud services and DevOps methodologies. The project focuses on implementing automated infrastructure provisioning, CI/CD pipelines, containerization using Docker, and Kubernetes orchestration through Azure Kubernetes Service (AKS). It also includes monitoring, logging, security integration, and AI-based cloud service implementation to improve application efficiency and reliability. The project mainly covers cloud deployment automation, Infrastructure-as-Code (IaC) using Terraform, container management, DevSecOps practices, role-based access control, and disaster recovery planning. Monitoring and observability tools such as Azure Monitor and Application Insights are used to track system performance and detect failures. The implementation is designed to demonstrate scalable deployment, automated operations, and secure cloud management in a controlled environment.However, the project boundaries are limited to Azure cloud infrastructure and the selected DevOps tools used during implementation. Advanced enterprise-level features such as multi-cloud integration, large-scale distributed systems, advanced AI model training, and high-volume production traffic handling are outside the scope of this project. The project is intended mainly for educational and demonstration purposes and may require additional enhancements for full-scale enterprise deployment.

.

1.4 STAKEHOLDERS AND END USERS

Stakeholders: The stakeholders of this project include project developers, DevOps engineers, cloud administrators, system administrators, security teams, and organizational management. These stakeholders are responsible for designing, deploying, managing, and monitoring the cloud infrastructure and application environment. They play an important role in decision-making, maintaining security policies, ensuring application availability, and improving overall system performance through automation and cloud technologies.

End Users: The end users are the individuals or organizations who access and use the deployed application services. They benefit from faster application performance, secure access, improved availability, and reliable cloud operations. End users experience seamless software updates, reduced downtime, and enhanced user experience due to automated deployment, monitoring, and scalable cloud infrastructure implemented in the project**.**

1.5 TECHNOLOGIES USED

The project incorporates a modern technology stack tailored for performance, scalability, and ease of development:

Table 1. Frontend Technologies

React	Used for developing the frontend user interface
TypeScript	Provides type safety and efficient application development
T**ailwind**	Used for responsive and modern UI design

Table 2: Backend Technologies

Go (Golang)	Used for building scalable backend APIs
Gin Framework	Lightweight HTTP web framework for API development
GORM	ORM library for database operations and migrations
PostgreSQL	Relational database for structured data storage
Blob Storage	Used for storing files and application data

Table 3: Cloud and Infrastructure

Microsoft Azure	The primary cloud provider for hosting and managing services.
Azure Kubernetes Service (AKS)	For container orchestration with auto-scaling.
Docker	Containerization of applications
Terraform	Infrastructure-as-Code (IaC) automation

Azure Container Registry (ACR)	Stores and manages Docker container images

Table 4: AI and Machine Learning

Azure AI Services	AI-based intelligent processing and automation
CI/CD Pipelines	Automated build, test, and deployment processes
DevSecOps Tools	Integration of security into DevOps workflow

Table 5: DevOps and CI/CD

GitHub Actions	Source code management and CI/CD pipelines
Azure Monitor	Monitoring and performance tracking
Application Insights	For command-line management of Azure resources.
RBAC	Secure access control management
SonarCloud / CodeQL	Security analysis and vulnerability scanning

1.6 ORGANIZATION OF THE REPORT

This report is structured to provide a complete and logical documentation of the entire project lifecycle.

Chapter 1 introduces the problem context, project objectives, scope, stakeholders, technologies, and report structure.

Chapter 2 presents a detailed examination of the system design and architecture, including requirements, solution overview, cloud strategy, infrastructure specifications, and service mappings.

Chapter 3 delves into the DevOps implementation, covering CI/CD pipelines, Terraform automation, Docker containerization, orchestration principles, and AI service integration. Chapter 4 focuses on operational excellence and security, discussing DevSecOps practices, monitoring, access control, and deployment strategies.

Chapter 5 presents the results of the implementation, including summaries, challenges overcome, performance metrics, cost analysis, and key learnings.

Chapter 6 concludes the report with a summary of achievements and a roadmap for future enhancements.

The document concludes with references and appendices containing configuration details and system screenshots.

CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

Functional Requirements

The Cloud-Based File Upload and Sharing App must provide a complete and secure user authentication system that allows new users to register with a valid email address and password, and existing users to log in and log out securely. The authentication system must be powered by a cloud-managed service such as Firebase Authentication or Supabase Auth, which handles session tokens, password resets, and account security automatically. Each user account must be uniquely identified so that all uploaded files and metadata are associated exclusively with that user, ensuring complete data separation between accounts.The application must support file uploads in multiple formats including PDFs, images (PNG, JPG, JPEG), and commonly used document types.

The upload interface must display real-time progress indicators so users are informed of the upload status at all times. Once uploaded, files must be stored securely in a cloud storage bucket such as Firebase Storage or AWS S3, and their metadata including file name, file type, file size, owner, and upload date must be recorded in a cloud database such as Firestore or MongoDB Atlas. Users must be able to view all their uploaded files in an organized dashboard with inline previews for supported formats.The system must provide a real-time search functionality that allows users to filter and find files by name, type, or upload date quickly and efficiently.

Users must be able to delete any of their uploaded files, with the deletion operation removing both the physical file from cloud storage and its corresponding metadata record from the database simultaneously. Additionally, the application must allow users to generate unique, shareable public links for any uploaded file, enabling others to access or download the file directly through the link without requiring an account or login.

Non-Functional Requirements

The application must be highly scalable and capable of handling increasing numbers of users and file uploads without degradation in performance. By utilizing cloud infrastructure such as AWS S3 or Firebase Storage, the system must automatically scale storage and compute resources in response to demand, eliminating the need for manual server provisioning. The cloud database must also support concurrent read and write operations efficiently, ensuring that multiple

users can upload, search, and manage files simultaneously without conflicts or delays.The system must maintain a high level of security and data privacy at all times. All data transmission between the client and server must be encrypted using HTTPS to prevent unauthorized interception.

User authentication must enforce strong password policies and token-based session management to protect accounts from unauthorized access. File access must be strictly controlled at the user level, ensuring that no user can view, download, or delete files belonging to another user unless a shareable link has been explicitly generated by the file owner.The application must deliver consistent and fast response time s across all operations including login, file upload, file listing, search, and link generation, regardless of the user's device or network speed.

The user interface must be responsive and compatible with all modern web browsers including Chrome, Firefox, Safari, and Edge, and must function correctly on both desktop and mobile devices without requiring any local software installation. The system must also be maintainable, with a clean and modular codebase that follows industry-standard development practices, making it easy to update, debug, and extend with new features in the future

2.2 PROPOSED SOLUTION OVERVIEWS

The proposed solution is a full-stack, cloud-native web application designed to provide users with a complete and secure file management experience entirely over the internet. The application serves as a lightweight personal file storage and sharing platform — similar in concept to a simplified version of Dropbox or Google Drive — but built from scratch using modern, industry-standard technologies to demonstrate real-world cloud computing principles. The system is designed to be accessible from any modern web browser without requiring any software installation, making it highly portable and user-friendly across a wide range of devices including desktops, laptops, and mobile phones.

The frontend of the application is developed using **React.js**, a popular component-based JavaScript library that enables the creation of dynamic and responsive user interfaces. The React frontend manages application state efficiently, providing users with real-time feedback during file uploads through progress indicators, and dynamically updating the file dashboard as files are added, deleted, or searched. The interface is designed to be clean, intuitive, and responsive, ensuring a consistent experience across different screen sizes and browsers. For developers preferring a simpler approach, a plain HTML, CSS, and JavaScript implementation is also available as an alternative frontend option.

The backend of the application is powered by **Node.js** combined with the **Express.js** framework, which together provide a lightweight and efficient server-side environment for handling API routes, processing file uploads, managing authentication logic, and communicating with cloud services. Express.js enables the definition of clean RESTful API endpoints that the frontend communicates with to perform all core operations such as user registration, login, file upload, file retrieval, file deletion, and shareable link generation. The backend acts as the central processing layer that connects the user interface with the underlying cloud infrastructure securely and efficiently.

For cloud services, the application offers multiple flexible options depending on the developer's familiarity and requirements. **Firebase** serves as an all-in-one solution providing Firebase Authentication for user management, Firebase Storage for file storage, and Firestore as the cloud database — making it the fastest and most beginner-friendly option to set up and deploy. Alternatively, **AWS S3** can be used for scalable object storage combined with **MongoDB Atlas** as a fully managed cloud database, offering greater control and demonstrating enterprise-level cloud architecture patterns. **Supabase** is also supported as an open-source Firebase alternative, providing built-in authentication alongside PostgreSQL-based database services.

All uploaded files are stored remotely in cloud storage buckets rather than on a local server disk, ensuring durability, redundancy, and global accessibility. File metadata including the file name, type, size, owner identity, upload timestamp, and download count is stored separately in the cloud database, enabling fast querying, search, and management operations without accessing the actual file content. User authentication is

handled entirely by cloud-managed auth services, which provide secure token-based session management, password encryption, and account recovery features out of the box, removing the need to build these complex security mechanisms manually.

The complete application is deployed to the cloud using hosting platforms such as **Vercel**, **Netlify**, or **Firebase Hosting**, which provide globally distributed content delivery, automatic HTTPS, and continuous deployment pipelines. This means the application is live on a publicly accessible URL, can be demonstrated from any device with an internet connection, and updates can be pushed to production automatically whenever changes are made to the codebase. Overall, the proposed solution effectively combines modern frontend development, backend API design, and cloud service integration into a single cohesive project that clearly demonstrates the practical advantages of cloud computing including scalability, accessibility, security, and cost-effectiveness.

2.3 CLOUD DEPLOYMENT STATERGY

The application follows a fully cloud-native deployment strategy, meaning every component of the system — from the user interface to the backend logic, file storage, and database — is hosted and managed on cloud platforms rather than on any local or dedicated physical server. This approach ensures that the application is globally accessible, highly available, and easy to maintain without requiring manual infrastructure management. By distributing responsibilities across specialized cloud services, each layer of the application benefits from the reliability, scalability, and security guarantees provided by industry-leading cloud providers.The frontend, built using React.js, is deployed to hosting platforms such as **Vercel** or **Netlify**, both of which offer globally distributed content delivery networks (CDN) that serve the application from servers closest to the user's geographic location, minimizing load times and latency. These platforms also support automatic continuous integration and continuous deployment (CI/CD) pipelines, meaning that whenever a developer pushes updated code to the main branch of the GitHub repository, the platform automatically builds and deploys the latest version of the frontend without any manual intervention. This significantly speeds up the development cycle and ensures users always have access to the most recent version of the application.The backend, developed using Node.js and Express.js, is deployed on cloud platforms such as **Firebase Cloud**

Functions or a cloud-based virtual machine, ensuring that API endpoints remain reliably available to handle incoming requests from the frontend at all times. Firebase Cloud Functions in particular offer a serverless execution model, meaning the backend code runs only when triggered by an API request and scales automatically based on traffic, eliminating the need to manage or pay for idle server resources. This serverless approach is both cost-efficient and highly scalable, making it well-suited for applications with variable or unpredictable traffic patterns. File storage is handled by **Firebase Storage** or **Amazon Web Services S3 (AWS**

S3), both of which are enterprise-grade object storage services designed to store and retrieve any amount of data reliably and securely. Files uploaded by users are stored in cloud buckets with unique identifiers, ensuring they are accessible via secure URLs at any time from anywhere in the world. These storage services provide built-in redundancy by automatically replicating data across multiple physical locations, protecting against data loss in the event of hardware failures. Access to stored files is controlled through security rules and authentication tokens, ensuring that only authorized users can access private files.

File metadata is managed through fully cloud-managed database services such as **Firestore** (part of the Firebase platform) or **MongoDB Atlas**, which provide fast, scalable, and real-time data storage and retrieval capabilities. These databases automatically handle replication, backups, and scaling, removing the burden of database administration from the developer. Authentication across the entire application is managed by **Firebase Auth** or **Supabase Auth**, which handle user session tokens, password encryption, account recovery, and security rules automatically .

2.4 INFRASTRUCTURE REQUIREMENTS

The infrastructure of the Cloud-Based File Upload and Sharing App is designed to be entirely cloudbased, meaning no dedicated physical servers or on-premise hardware are required to develop, deploy, or operate the application. This approach significantly reduces setup costs, eliminates maintenance overhead, and allows the system to scale automatically in response to user demand. The infrastructure is composed of several interconnected layers, each fulfilled by a specific cloud service or development tool, working together to deliver a reliable and performant application.

On the client side, the application requires a modern web browser such as Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari to access and interact with the frontend interface. The development environment requires a machine with a minimum of 4 GB of RAM, although 8 GB or more is recommended for a smoother development experience, particularly when running the React development server alongside other tools. Node.js must be installed on the development machine to run the backend server locally during development and to manage project dependencies through the Node Package Manager (npm). No special hardware or operating system is required, as the development environment is compatible with Windows, macOS, and Linux.

The frontend layer is built using React.js as the primary framework, supported by standard web technologies including HTML, CSS, and JavaScript. The backend layer relies on Node.js and Express.js to create and manage RESTful API endpoints that handle all server-side logic including file processing, authentication verification, and database communication. Together, these two layers form the core application stack that developers build and test locally before deploying to the cloud.

For cloud infrastructure, the application requires a cloud storage service such as Firebase Storage or AWS S3 to store uploaded files remotely in scalable and durable object storage buckets. A cloud database service such as Firestore, MongoDB Atlas, or Supabase PostgreSQL is required to store and manage file metadata, user records, and application data with fast query performance. A cloud authentication service such as Firebase Auth or Supabase Auth is required to manage user accounts, session tokens, and access control securely without building custom authentication logic from scratch. For deployment and hosting, the application requires a frontend hosting platform such as Vercel, Netlify, or Firebase Hosting to serve the React application over a globally distributed CDN with automatic HTTPS and CI/CD support. The backend requires a serverless or cloud hosting environment such as Firebase Cloud Functions or a cloud VM to ensure API availability at all times. Finally, Git and GitHub are required for version control and source code management, enabling collaborative development, code tracking, and automated deployment triggers. Together, this infrastructure stack is cost-effective, beginner-friendly, and fully capable of supporting a production-grade cloud application without the complexity of managing physical servers**.**

2.5. Azure Services Mapping and Justification

Table: Service Mapping – Cloud-Based File Upload and Sharing App

Requirement	Cloud Service	SKU/Tier	Purpose	Justification	Est. Monthly Cost (₹)
Frontend Hosting	Vercel / Netlify	Free Tier	Serve React UI globally	Free hosting, SSL, CDN	Free (Student Use)
Backend Logic & APIs	Firebase Cloud Functions / Node.js Server	Spark Plan	Handle API routes, file processing	Serverless, scalable, no idle cost	Free / ~500
Database	Firestore / MongoDB Atlas	Free Tier	Store file metadata, user records	Reliable, auto-backup, real-time sync	Free / ~800
File Storage	Firebase Storage / AWS S3	Free Tier / Standard	Store uploaded files (PDF, images, docs)	Scalable, durable, globally accessible	Free / ~300
Authentication	Firebase Auth / Supabase Auth	Free Tier	User registration, login, session management	Secure, token-based, easy integration	Free
Real-Time Search	Firestore Queries / Algolia	Free Tier	Filter files by name, type, date	Fast, efficient, low-latency results	Free / ~200
Shareable Links	Firebase Storage URLs	Standard	Generate public file access links	Simple, secure, no extra service needed	Included
Monitoring & Logs	Firebase Analytics / Vercel Logs	Standard	Track usage, errors, API health	Proactive maintenance, debugging	Free

CHAPTER 3 - DEVOPS IMPLEMENTATION

3.1 CONTINUOUS INTEGRATION AND DEPLOYMENT STATERGY

Continuous Integration and Continuous Deployment (CI/CD) is an important DevOps practice used in the development of the Cloud-Based File Upload and Sharing Application. The main objective of CI/CD is to automate the process of building, testing, and deploying the application so that new updates can be delivered quickly and reliably. In this project, whenever developers make changes to the source code, the updated code is automatically integrated into the main repository. This helps in identifying coding errors at an early stage and ensures that the application remains stable throughout the development lifecycle.

The Continuous Integration process involves the use of version control systems such as Git and platforms like GitHub to manage the project code. Each time code is pushed to the repository, automated workflows are triggered to check the quality of the application. These workflows perform tasks such as dependency installation, code compilation, and basic testing of frontend and backend components. By automating these activities, developers can reduce manual errors, improve collaboration, and maintain consistency in the development environment.

Continuous Deployment is used to automatically deploy the latest stable version of the application to a cloud hosting platform such as Vercel, Netlify, or Firebase Hosting. Once the testing phase is completed successfully, the updated application is published to the live server without requiring manual intervention. This deployment strategy ensures that users always have access to the latest features and improvements. It also reduces deployment time and increases the overall efficiency of the software delivery process.

The CI/CD strategy implemented in this project improves software reliability, scalability, and maintainability. Automated deployment pipelines help developers monitor application performance and quickly resolve issues when detected. In addition, cloud-based deployment enables the application to support multiple users efficiently while ensuring high availability and secure access. Thus, the integration of DevOps practices enhances the overall quality and performance of the Cloud-Based File Upload and Sharing Application.

3.2 TERRAFORM INFRASTRUCTURE-AS-CODE (IaC)

Terraform is an Infrastructure-as-Code (IaC) tool that enables developers to automate the provisioning and management of cloud infrastructure using configuration files. In the Cloud-Based File Upload and Sharing Application, Terraform is used to create and manage infrastructure resources efficiently without manual configuration. By defining infrastructure in code, the project ensures consistency, reliability, and repeatability

across development, testing, and production environments. This approach reduces human errors and simplifies this deployment process in modern DevOps practices.

The core infrastructure components managed through Terraform include cloud storage services, networking resources, virtual machines, authentication services, and hosting platforms. Terraform automates the setup of storage buckets for uploaded files, cloud servers for backend services, and security configurations required for secure communication. These infrastructure components are defined in reusable configuration files, making it easier to scale the application whenever user demand increases. As a result, the application becomes more flexible, scalable, and easier to maintain.

Terraform also supports backend management and container orchestration by integrating with technologies such as Docker and Kubernetes. Containers help package the application along with its dependencies, ensuring consistent execution across different environments. Kubernetes can be used to manage and orchestrate these containers efficiently by handling deployment, scaling, and monitoring automatically. In addition, Terraform provisions database services such as MongoDB Atlas or Firebase Firestore to securely store user information and file metadata. This automated database configuration improves performance, reliability, and data availability.

Another important feature of Terraform is its integration with Continuous Integration and Continuous Deployment (CI/CD) pipelines. Terraform scripts can be executed automatically during the deployment process whenever changes are made to the application code. CI/CD tools such as GitHub Actions, Jenkins, or GitLab CI can trigger infrastructure updates and application deployment without manual intervention. This automation speeds up software delivery, improves collaboration among developers, and ensures that infrastructure changes are properly version-controlled. Therefore, Terraform Infrastructure-as-Code plays a significant role in enhancing the efficiency, scalability, and maintainability of the Cloud-Based File Upload and Sharing Application.

3.3 CONTAINERIZATION STRATERGY (DOCKER)

Containerization is an important DevOps practice used to package applications along with their dependencies into lightweight and portable containers. In the Cloud-Based File Upload and Sharing Application, Docker is used as the primary containerization platform to ensure that the application runs consistently across different environments such as development, testing, and production. Docker eliminates compatibility issues by

packaging the frontend, backend, libraries, and runtime environment into a single container image. This makes the deployment process faster, more reliable, and easier to manage.

The application architecture can be divided into multiple containers based on functionality. For example, the React frontend, Node.js backend server, and database services can each run in separate Docker containers. Docker Compose is used to define and manage these multi-container services efficiently. By isolating components into individual containers, developers can update, test, and scale each service independently without affecting the entire application. This modular approach improves maintainability and supports microservice-based deployment strategies.

Docker also improves scalability and resource utilization in cloud environments. Containers are lightweight compared to traditional virtual machines because they share the host operating system kernel while maintaining isolated execution environments. This allows multiple containers to run efficiently on the same server with minimal overhead. In this project, Docker containers can be deployed on cloud platforms and integrated with orchestration tools such as Kubernetes for automatic scaling, monitoring, and load balancing. As user traffic increases, additional containers can be created dynamically to maintain application performance and availability.

Another major advantage of Docker is its seamless integration with CI/CD pipelines and DevOps workflows. Docker images can be automatically built, tested, and deployed whenever changes are pushed to the source code repository. This automation ensures faster software delivery and reduces deployment errors. Furthermore, Docker Hub or other container registries can be used to store and distribute container images securely. Therefore, the Docker-based containerization strategy enhances portability, consistency, scalability, and deployment efficiency in the Cloud-Based File Upload and Sharing Application.

3.4 KUBERNETES ORCHESTRATION

Kubernetes is an open-source container orchestration platform used to automate the deployment, management, scaling, and monitoring of containerized applications. In the Cloud-Based File Upload and Sharing Application, Kubernetes is used to manage Docker containers efficiently across cloud environments. Since the application consists of multiple services such as the frontend, backend, and database components, Kubernetes helps

coordinate these services and ensures smooth communication between them. This orchestration platform improves application reliability, scalability, and availability in production environments.

One of the major functions of Kubernetes is automated deployment and scaling of containers. Kubernetes organizes containers into units called pods and distributes them across available computing resources. When user traffic increases, Kubernetes automatically scales the application by creating additional container instances to handle the workload efficiently. Similarly, when demand decreases, unnecessary containers are removed to optimize resource usage. This automatic scaling capability helps maintain application performance while reducing operational costs in cloud environments.

Kubernetes also provides advanced features such as load balancing, self-healing, and service discovery. Load balancing distributes incoming user requests evenly across multiple containers to prevent server overload. The self-healing mechanism automatically restarts failed containers or replaces unhealthy pods without manual intervention. Service discovery enables different components of the application to communicate securely and efficiently within the cluster. These features ensure continuous availability and smooth operation of the Cloud-Based File Upload and Sharing Application even during failures or heavy traffic conditions.

Another important advantage of Kubernetes is its integration with DevOps and CI/CD workflows. Kubernetes supports automated deployment pipelines where updated Docker container images are deployed seamlessly after testing. Configuration files written in YAML are used to define application services, scaling rules, and networking policies. This infrastructure automation simplifies system management and improves deployment consistency across environments. Therefore, Kubernetes orchestration enhances the scalability, fault tolerance, automation, and operational efficiency of the Cloud-Based File Upload and Sharing Application.

3.5 GENAI INTEGRATION AND AZURE AI SERVICE MAPPING

Generative Artificial Intelligence (GenAI) integration enhances the functionality and intelligence of the Cloud-Based File Upload and Sharing Application by enabling automated content understanding and moderation features. In this project, advanced AI services are integrated to improve user experience, automate repetitive tasks, and ensure platform safety. Cloud-based AI APIs are utilized to process uploaded content efficiently without requiring complex local computation. The integration of GenAI technologies demonstrates how modern cloud applications can combine storage systems with intelligent AI-powered services.

One of the important features implemented in the project is AI-Generated Descriptions using the Google Gemini Pro Vision API. This AI model can analyze uploaded images and automatically generate meaningful textual descriptions based on image content. For example, when users upload photos or documents, the system can provide a short summary or caption describing the image. This functionality improves accessibility, enhances file organization, and helps users quickly identify uploaded content. The Gemini Pro Vision API uses advanced computer vision and natural language generation techniques to understand visual information and convert it into human-readable descriptions.

Another key integration is Content Safety Moderation using the Azure Content Safety API. This service helps detect and filter harmful, offensive, or inappropriate content uploaded by users. The API analyzes text and image content for categories such as violence, hate speech, explicit material, or abusive language. By implementing automated moderation, the application can maintain a secure and user-friendly environment while reducing the need for manual content review. This feature is especially important in cloud-based sharing platforms where user-generated content must be monitored continuously to ensure compliance with safety standards and ethical guidelines.

The integration of GenAI services and Azure AI technologies improves the overall intelligence, security, and usability of the application. AI-generated descriptions provide smart automation and better accessibility, while content safety moderation ensures responsible content management. These services can be integrated into the backend using REST APIs and processed in real-time whenever files are uploaded. Therefore, the use of Google Gemini Pro Vision API and Azure Content Safety API demonstrates the practical application of artificial intelligence in enhancing modern cloud-based systems and DevOps-driven applications.

CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

4.1 DevSecOps Integration

DevSecOps is a modern software development approach that integrates security practices into every stage of the DevOps lifecycle. In the Cloud-Based File Upload and Sharing Application, DevSecOps ensures that security is not treated as a separate process but as a continuous and automated part of development, testing, deployment, and operations. By incorporating security measures early in the development cycle, the project minimizes vulnerabilities, improves system reliability, and protects sensitive user data stored in the cloud environment.

The DevSecOps implementation in this project includes secure coding practices, automated vulnerability scanning, authentication management, and encrypted communication. During development, source code is continuously analyzed using security testing tools to identify coding weaknesses and potential threats. User authentication mechanisms such as secure login systems and token-based access control are implemented to prevent unauthorized access. Additionally, HTTPS protocols and encrypted cloud storage services are used to ensure secure data transmission and file protection.

Another important aspect of DevSecOps is the integration of automated security testing within the CI/CD pipeline. Whenever developers push code changes to the repository, automated workflows perform security checks alongside build and deployment tasks. These checks may include dependency vulnerability analysis, container security scanning, and configuration validation. By automating security processes, the application can quickly detect and resolve issues before deployment to production environments. This proactive approach reduces security risks and ensures continuous compliance with cloud security standards.

DevSecOps also enhances monitoring, incident management, and operational security in cloud environments. Continuous monitoring tools help detect suspicious activities, unauthorized access attempts, and system failures in real time. Security logs and alerts enable administrators to respond quickly to threats and maintain application stability. Through the integration of security with DevOps practices, the Cloud-Based File Upload and Sharing Application achieves improved data protection, operational efficiency, and secure cloud service management. Therefore, DevSecOps plays a vital role in ensuring the reliability, scalability, and trustworthiness of the system.

\

4.2 MONITORING AND OBSERVABILITY

Monitoring and observability are essential practices in cloud operations that help maintain the performance, reliability, and availability of applications. In the Cloud-Based File Upload and Sharing Application, monitoring is used to continuously track the health and performance of servers, containers, databases, and cloud services. Observability extends this capability by providing deeper insights into system behavior through logs, metrics, and traces. Together, these practices help administrators quickly identify issues, analyze system performance, and ensure smooth operation of the application.

The monitoring process involves collecting real-time data related to CPU usage, memory consumption, network traffic, file upload activity, and server response time. Tools such as Prometheus, Grafana, or cloud-native monitoring services can be used to visualize and analyze this data through dashboards and alerts. By continuously observing system metrics, administrators can detect unusual activities, performance bottlenecks, or service failures before they impact users. This proactive monitoring approach improves system stability and enhances user experience.

Observability focuses on understanding the internal state of the application by analyzing logs, distributed traces, and event data. Application logs record important activities such as user logins, file uploads, API requests, and error messages. Distributed tracing helps track requests across multiple services including frontend, backend, cloud storage, and databases. This detailed visibility enables developers to diagnose problems efficiently and optimize application performance. Observability is especially important in cloud-native and containerized environments where multiple services interact dynamically.

Monitoring and observability also play a critical role in security and operational management. Real-time alerts notify administrators about suspicious activities, failed login attempts, or sudden increases in resource usage. Historical data analysis helps identify long-term trends and supports capacity planning for future scalability. Integration with DevOps and Kubernetes environments further improves automated management and incident response. Therefore, monitoring and observability enhance the reliability, performance, security, and maintainability of the Cloud-Based File Upload and Sharing Application in modern cloud environments.

4.3 ACCESS CONTROL

Access control is an important security mechanism used to regulate and manage user permissions within the Cloud-Based File Upload and Sharing Application. The main objective of access control is to ensure that only authorized users can access, upload, modify, share, or delete files stored in the cloud environment. By implementing proper authentication and authorization techniques, the system protects sensitive user data from unauthorized access and cyber threats. This security approach helps maintain privacy, confidentiality, and integrity of stored information.

The application uses secure user authentication methods such as email-password login and token-based authentication to verify user identities. Cloud authentication services like Firebase Authentication or OAuth-based systems can be integrated to provide secure and reliable user management. Once authenticated, users receive secure session tokens that allow them to access application resources safely. Password encryption and secure communication protocols such as HTTPS further enhance security by protecting login credentials and user sessions from interception.

Authorization mechanisms are implemented to define different levels of access permissions for users. Each user can only view and manage their own uploaded files unless a file is intentionally shared through generated public links. Role-based access control (RBAC) can also be applied to assign different privileges to administrators and regular users. For example, administrators may have permission to monitor system activity and manage user accounts, while normal users can upload, download, search, and share files within their authorized scope. This structured permission management improves overall system security and operational control.

Access control also supports auditing and monitoring activities within the application. Logs are maintained to record user actions such as file uploads, downloads, sharing operations, and deletion requests. These logs help administrators track suspicious activities and ensure compliance with security policies. Integration with cloud security services and DevSecOps practices further strengthens protection against unauthorized access attempts. Therefore, access control plays a vital role in safeguarding user data, maintaining system integrity, and ensuring secure cloud operations in the Cloud-Based File Upload and Sharing Application.

4.4 BLUE–GREEN DEPLOYMENT & DISASTER RECOVERY PLANNING

Blue–Green Deployment is a modern deployment strategy used in cloud environments to reduce downtime and ensure smooth application updates. In the Cloud-Based File Upload and Sharing Application, two identical production environments known as “Blue” and “Green” are maintained. One environment remains active and serves users, while the other is used for testing and deploying new updates. Once the updated version is verified

successfully, user traffic is switched from the active environment to the updated one. This approach minimizes service interruptions and allows quick rollback if any issues are detected after deployment.

The Blue–Green deployment strategy improves reliability, availability, and user experience during software releases. Since updates are tested in a separate environment before going live, the risk of deployment failures affecting end users is significantly reduced. In this project, cloud hosting platforms and container orchestration tools such as Kubernetes can manage traffic routing between the two environments efficiently. Load balancers help direct users to the correct environment while ensuring continuous access to application services. This deployment model supports DevOps practices by enabling faster and safer software delivery.

Disaster Recovery Planning (DRP) is another critical aspect of cloud operations and security. Disaster recovery refers to the process of restoring application services and data after unexpected failures such as server crashes, cyberattacks, hardware failures, or natural disasters. In the Cloud-Based File Upload and Sharing Application, disaster recovery strategies include automated data backups, cloud replication, and failover mechanisms. File data and database records are stored securely in cloud environments with backup copies maintained in separate locations to ensure data availability and protection.

The combination of Blue–Green deployment and disaster recovery planning enhances business continuity and system resilience. Continuous backups, monitoring systems, and automated recovery procedures help minimize downtime and data loss during failures. Cloud platforms provide scalable and distributed infrastructure that supports rapid recovery and high availability. By integrating deployment safety mechanisms with disaster recovery strategies, the application achieves improved reliability, fault tolerance, and operational stability. Therefore, these cloud operational practices play a significant role in maintaining secure and uninterrupted service delivery in the Cloud-Based File Upload and Sharing Application.

CHAPTER 5 - RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The Cloud-Based File Upload and Sharing App was successfully implemented as a full-stack web application using React.js for the frontend and Node.js with Express.js for the backend, integrated with cloud services including Firebase Authentication, Firebase Storage, and Firestore as the primary database. The implementation followed a structured development approach, beginning with the setup of the development environment, followed by the creation of the user authentication system, the file upload and storage module, the file management dashboard, and finally the shareable link generation feature. Each component was developed and tested independently before being integrated into the complete system, ensuring that individual modules functioned correctly before the full application was assembled and deployed to the cloud.The user authentication module was implemented using Firebase Authentication, which provided a secure and reliable system for user registration, login, and session management without requiring custom authentication logic. Upon successful login, users are directed to a personalized dashboard that displays all files associated exclusively with their account, retrieved in real time from the Firestore database. The authentication system correctly enforces access control, ensuring that no user can access, view, or manage files belonging to another user, and that all sessions are managed securely through encrypted tokens that expire after a defined period of inactivity. The file upload and storage module was implemented to support multiple file formats including PDFs, images (PNG, JPG, JPEG), and common document types. Files are uploaded directly from the browser to Firebase Storage buckets using secure upload APIs, with real-time progress indicators displayed to the user throughout the upload process. Upon successful upload, the file's metadata including the file name, type, size, owner UID, and upload timestamp is automatically recorded in the Firestore database, making the file immediately visible in the user's dashboard without requiring a page refresh. The delete functionality was also implemented to simultaneously remove both the file from Firebase Storage and its corresponding metadata record from Firestore, ensuring complete and consistent data cleanup.The shareable link generation feature and real-time search functionality were successfully implemented as the final components of the application. The link generation feature creates a unique, publicly accessible URL for any selected file, allowing the file owner to share it with others who can access or download the file without needing an account. The real-time search feature dynamically filters the file list based on the user's input, matching results by file name, type, or upload date instantly without requiring a full page reload. The completed application was deployed successfully to Vercel for frontend hosting and Firebase for backend and cloud services, making

it fully accessible via a live public URL that can be demonstrated from any device with an internet connection.

.

5.2 CHALLENGES FACED AND RESOLUTIONS

One of the primary challenges encountered during development was configuring Firebase security rules to correctly restrict file access based on user identity while still allowing shareable links to be accessed publicly by anyone with the URL. Initially, overly restrictive rules blocked public access to shared files entirely, while overly permissive rules exposed all files to unauthenticated users. This was resolved by carefully defining conditional security rules in Firebase Storage and Firestore that distinguish between private user-owned files requiring authentication and explicitly shared files granted public read access through a dedicated sharing flag stored in the database.

Another significant challenge was handling large file uploads reliably on slower network connections, where upload interruptions and timeouts were observed during testing. The initial implementation did not account for network instability, causing failed uploads to leave incomplete records in Firestore without a corresponding file in Firebase Storage. This was resolved by implementing error handling logic that detects upload failures, rolls back any partial database writes, notifies the user with a clear error message, and provides the option to retry the upload, ensuring data consistency between the storage bucket and the database at all times.

Integrating real-time search functionality with Firestore presented a technical challenge, as Firestore does not natively support full-text search queries in the way that traditional relational databases do. This was resolved by storing a lowercase version of each file name as a separate field in the database and performing prefix-based range queries against this field, enabling effective partial-name search functionality. Deploying the Node.js backend to Firebase Cloud Functions also introduced cold start latency issues, which were partially mitigated by optimizing the backend initialization code, minimizing startup dependencies, and impleme

5.3 PERFORMANCE OR COST OBSERVATION

The application demonstrated strong overall performance during testing, with file upload speeds being primarily dependent on the user's internet connection speed rather than any bottleneck within the application itself. For typical file sizes ranging from a few kilobytes to several megabytes, upload operations completed within a few seconds on a standard broadband connection, with real-time progress indicators providing continuous feedback throughout the process. File retrieval and dashboard loading operations were consistently fast, with Firestore database queries returning metadata records in under one second for user accounts containing up to several hundred files, demonstrating that the chosen architecture is well-suited for the expected scale of the application.

The search functionality performed efficiently during testing, returning filtered results almost instantaneously as users typed their queries, thanks to real-time state management in React combined with efficient Firestore prefix-range queries. The shareable link generation feature operated reliably, producing publicly accessible URLs within milliseconds of the user triggering the action, with no noticeable delay or degradation observed during repeated testing. The delete operation also performed consistently, with simultaneous removal of the file from Firebase Storage and its metadata from Firestore completing successfully in all test cases without leaving orphaned records in either system.

From a cost perspective, the application was developed and tested entirely within the free-tier limits offered by Firebase, Vercel, and MongoDB Atlas, making it a zero-cost project for academic and demonstration purposes. Firebase's free Spark Plan provides generous allowances including 1 GB of Cloud Storage, 50,000 database reads and 20,000 writes per day on Firestore, and up to 10,000 authentication operations per month, all of which were sufficient to support the full development and testing phases. The serverless backend architecture further contributed to cost efficiency, as charges are only incurred for actual function execution time rather than continuous server uptime, making the overall cost structure transparent, predictable, and well-aligned with the project's goals

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

The development of this project provided the team with extensive hands-on experience in full-stack web development using modern, industry-standard technologies. Working with React.js deepened understanding of component-based architecture, state management, and asynchronous operations such as file uploads and real-time database queries. Building the backend with Node.js and Express.js reinforced knowledge of

RESTful API design, middleware configuration, and secure communication between the server and cloud services, all of which are directly applicable to real-world software development roles.

Integrating Firebase services gave the team firsthand experience with cloud platform configuration, security rule management, and the practical trade-offs involved in choosing between different cloud service options. The team gained a deep understanding of how cloud-managed services abstract away complex infrastructure concerns such as server provisioning, database replication, and SSL certificate management, allowing developers to focus entirely on application logic and user experience. Working with alternative backend options such as AWS S3 and MongoDB Atlas further broadened exposure to enterprise-level cloud architecture patterns and reinforced the importance of designing cloud-provider-agnostic applications.

Each team member contributed meaningfully to different aspects of the project, with responsibilities distributed across frontend development, backend API design, cloud service integration, testing, and documentation. The frontend developer focused on building the React interface, implementing the file dashboard, search functionality, and upload progress indicators, while the backend developer configured the Node.js server and Firebase services. Collaborative problem-solving during the challenges described in section 5.2 was particularly valuable, demonstrating the team's ability to work together effectively under technical pressure and arrive at well-reasoned solutions through discussion and experimentation.

CHAPTER 6 - CONCLUSION AND FUTURE WORKS

6.1 CONCLUSION

The Cloud-Based File Upload and Sharing App was successfully designed, developed, and deployed as a fully functional full-stack web application that demonstrates the practical application of modern cloud computing principles. The application achieves all of its primary objectives, providing users with a secure, accessible, and intuitive platform for uploading, managing, and sharing files entirely through a browser-based interface without requiring any local software installation or dedicated server infrastructure. By integrating Firebase Authentication, Firebase Storage, Firestore, and Vercel hosting, the project effectively showcases how cloud-managed services can be combined to deliver a production-grade application with minimal infrastructure overhead and zero upfront cost.

The project successfully demonstrated that cloud-native applications offer significant advantages over traditional server-based architectures in terms of scalability, availability, cost-efficiency, and ease of deployment. The automatic scaling capabilities of Firebase Storage and Firestore ensured consistent performance under varying levels of concurrent usage, while the serverless backend architecture eliminated the need for continuous server management. The CI/CD pipeline established through Vercel demonstrated how modern deployment practices can accelerate development cycles and reduce the risk of production errors, reinforcing the value of adopting cloud-native workflows from the earliest stages of a project.

From an academic perspective, the project provided the team with comprehensive hands-on experience across the full spectrum of modern web application development, from frontend UI design and state management to backend API development, cloud service integration, and production deployment. The challenges encountered during development provided valuable learning opportunities that deepened technical understanding and problem-solving capabilities across the entire team. In conclusion, the project stands as a well-rounded, technically sound, and practically relevant piece of work that meets all defined requirements and serves as a strong portfolio demonstration of the team's ability to design and deliver a complete cloud-based software solution.

6.2 FUTURE WORKS

While the current implementation successfully fulfills all defined project requirements, there are numerous opportunities for enhancement that could significantly expand the application's functionality and usability in future development phases. One of the most impactful planned enhancements is the introduction of folder

organization and file categorization, allowing users to create custom folder structures, group related files together, and navigate their stored content more efficiently. This feature would greatly improve the user experience for accounts with large numbers of files and would bring the application closer in functionality to established platforms such as Google Drive and Dropbox.

Another important area of future development is the implementation of **team collaboration and file sharing features**, allowing multiple users to collaborate on shared folders and control access permissions at a granular level. This would include the ability to invite registered users to access specific folders with defined permission levels such as view-only or full edit access, enabling the application to serve as a lightweight team file management tool. Integration with communication platforms such as Slack or Microsoft Teams could further enhance collaboration by allowing users to share file links directly within existing team workflows without leaving the application.

From an infrastructure and security perspective, future enhancements could include **file versioning** to maintain a history of all previous file versions, **storage quota management** to enforce per-user storage limits, and **end-to-end encryption** for sensitive files to ensure that even cloud service providers cannot access encrypted content. AI-powered features such as automatic file tagging, full-text search using Algolia or Elasticsearch, and OCR-based content indexing for scanned documents could be integrated to significantly improve file discovery and organization. Expanding the backend to support multi-region deployments would further improve availability and reduce latency for users across different geographic locations, positioning the application for enterprise-grade production use at scale.